

UNIVERSIDAD CATÓLICA DE CUENCA



**UNIDAD ACADÉMICA DE INFORMÁTICA, CIENCIAS DE LA COMPUTACIÓN,
E INNOVACIÓN TECNOLÓGICA.**

SISTEMAS COMPUTACIONALES

DISEÑO Y GESTION DE BASE DE DATOS

NOMBRE: JOAQUIN ASTUDILLO, GUSTAVO ACURIO, JORDY DUTAN

DOCENTE: JUAN CUENCA

FECHA: 19/07/26

1. Resumen ejecutivo

El presente informe documenta el desarrollo del Sistema Inteligente de Gestión de Alícuotas, una solución de base de datos orientada a la administración financiera de una urbanización. El sistema permite gestionar urbanizaciones, propietarios, viviendas, períodos de cobro, alícuotas, pagos, egresos, caja, auditoría, reportes, servicios REST y consultas en lenguaje natural.

La solución fue implementada sobre Oracle Database local y publicada mediante Oracle REST Data Services (ORDS). ORDS actúa como gateway HTTPS para Oracle Database y permite exponer datos, consultas SQL y lógica PL/SQL como servicios REST que devuelven respuestas en formato JSON [1]. Además, los servicios creados pueden visualizarse desde Oracle Database Actions en la sección REST, específicamente en módulos, plantillas y manejadores [2].

El sistema fue probado con Postman y desde la interfaz de Oracle Database Actions. Postman se utilizó para enviar solicitudes HTTP, inspeccionar respuestas y validar el comportamiento de los endpoints REST [7]. Finalmente, el proyecto fue versionado y documentado en GitHub, donde se publicaron scripts, plantillas, evidencias y documentación [8].

Componente	Resultado implementado
Modelo relacional	Tablas para urbanización, propietarios, viviendas, períodos, alícuotas, pagos, egresos, caja, auditoría e IA.
SQL avanzado	Consultas de deuda, recaudación, caja, top deudores, deudores por filtros y reportes financieros.
PL/SQL	Procedimientos y funciones para registrar pagos, generar alícuotas, registrar egresos y procesar lenguaje natural.
REST/ORDS	Módulo mod_alicuotas_api con plantillas y manejadores GET, POST, PUT y DELETE.
JSON	Uso de respuestas JSON y vistas JSON para reportes estructurados.
IA/NLQ	Módulo simulado de lenguaje natural que interpreta preguntas y genera SQL.
GitHub	Repositorio con estructura profesional, README, scripts, evidencias y plantillas.

2. Introducción

En una urbanización o conjunto residencial, las alícuotas representan los valores mensuales que pagan las viviendas para cubrir gastos comunes como mantenimiento, seguridad, limpieza, jardinería, luminarias o reparación de accesos. Una gestión manual de estos procesos puede generar errores en el control de pagos, deudas, egresos y saldo de caja.

Por esta razón, se desarrolló una solución basada en base de datos Oracle que centraliza la información administrativa y financiera. El proyecto no se limita al almacenamiento de datos: integra lógica de negocio en PL/SQL, auditoría con triggers, generación de reportes, API REST con ORDS, pruebas con Postman, documentación en GitHub y un módulo de consultas en lenguaje natural.

El enfoque utilizado permite demostrar que Oracle Database puede funcionar como núcleo de datos, motor de reglas, generador de JSON, publicador de servicios REST y repositorio de evidencias del sistema.

3. Objetivos

3.1 Objetivo general

Diseñar e implementar un sistema inteligente de gestión de alcuotas para una urbanización, utilizando Oracle Database, SQL avanzado, PL/SQL, ORDS, servicios REST, JSON y un módulo de consultas en lenguaje natural.

3.2 Objetivos específicos

- Modelar una base de datos relacional para representar urbanizaciones, propietarios, viviendas, períodos, alcuotas, pagos, egresos y movimientos de caja.
- Implementar consultas SQL avanzadas para obtener reportes administrativos y financieros.
- Crear procedimientos, funciones y triggers en PL/SQL para automatizar pagos, egresos, auditoría y generación de alcuotas.
- Publicar una API REST con ORDS para permitir operaciones CRUD y reportes desde clientes externos.
- Probar la API mediante Postman y Oracle Database Actions REST.
- Incorporar un módulo IA/NLQ que reciba preguntas en lenguaje natural, genere SQL y guarde evidencia.
- Documentar el proyecto en GitHub con scripts, plantillas, capturas y un README profesional.

4. Alcance del proyecto

El sistema cubre el ciclo principal de administración de alcuotas, desde la creación de propietarios y viviendas hasta la generación de obligaciones mensuales, registro de pagos, control de egresos y consulta de reportes. El enfoque de seguridad aplicado evita eliminaciones físicas en registros financieros sensibles, usando estados lógicos como INACTIVO, CERRADO o ANULADA para preservar trazabilidad.

Incluido	Descripción
Gestión de propietarios	Crear, listar, consultar por ID, actualizar e inactivar propietarios.
Gestión de viviendas	Crear, listar, actualizar e inactivar viviendas asociadas a propietarios.
Gestión de períodos	Crear períodos mensuales y cambiar su estado operativo.
Gestión de alcuotas	Crear alcuotas individuales o generarlas masivamente por mes.
Pagos	Registrar pagos y actualizar el estado de la alcuota.
Egresos	Registrar gastos comunes y afectar el saldo de caja.
Caja	Consultar ingresos, egresos y saldo financiero.
Auditoría	Guardar eventos importantes mediante triggers.
IA/NLQ	Interpretar preguntas frecuentes y generar SQL relacionado.
API REST	Consumir operaciones desde Postman o frontend básico.

5. Fundamentación técnica

5.1 Oracle Database y PL/SQL

Oracle Database permite crear objetos relacionales, vistas, funciones, procedimientos y triggers. PL/SQL permite centralizar lógica de negocio dentro de la base de datos mediante subprogramas reutilizables. Oracle describe los procedimientos y funciones como tipos de subprogramas PL/SQL, y los paquetes como objetos que agrupan elementos relacionados [4].

En este proyecto, PL/SQL se utilizó para registrar pagos, generar alícuotas, registrar egresos, calcular deudas y procesar preguntas de lenguaje natural.

5.2 ORDS y servicios REST

Oracle REST Data Services permite exponer datos y lógica de Oracle Database mediante servicios REST. La documentación oficial indica que ORDS puede ejecutar sentencias SQL contra la base de datos y devolver los resultados al cliente en formato JSON [1].

Asimismo, Oracle Database Actions permite crear y administrar servicios REST desde una interfaz gráfica mediante módulos, plantillas y manejadores [2]. En este proyecto, los servicios se generaron mediante scripts ORDS, pero se verificaron visualmente desde la interfaz REST de Database Actions.

5.3 JSON en Oracle

Oracle Database incluye funciones SQL/JSON para generar datos JSON desde información relacional. La función `JSON_OBJECT` construye objetos JSON a partir de pares clave-valor evaluados por expresiones SQL [5], y `JSON_ARRAYAGG` permite construir arreglos JSON agregados [6].

En el proyecto se emplearon vistas JSON para representar información de propietarios, viviendas, alícuotas, deudores y reportes financieros en un formato compatible con aplicaciones web y clientes REST.

5.4 Select AI y módulo NLQ local

Oracle Select AI permite interactuar con la base de datos mediante lenguaje natural, generando, ejecutando o explicando consultas según la acción configurada [9]. Sin embargo, en este proyecto se trabajó en entorno local sin perfil activo de Select AI, por lo que se desarrolló un módulo NLQ simulado en PL/SQL. Este módulo mantiene el flujo académico: pregunta natural, interpretación, SQL generado y evidencia guardada.

6. Arquitectura del sistema

La arquitectura implementada separa el cliente de consumo, los servicios REST y la base de datos. El navegador, Postman o el frontend básico consumen endpoints publicados por ORDS; ORDS ejecuta consultas SQL o bloques PL/SQL; y Oracle Database conserva los datos, reglas, auditoría y reportes.

Cliente / Navegador / Postman / Frontend



Oracle REST Data Services (ORDS)



Módulo REST: mod_alicuotas_api

Base path: /api/



Esquema Oracle: PROY_ALICUOTAS_ASTUDILLO



Tablas, vistas, funciones, procedimientos, triggers, JSON e IA/NLQ

Nivel	Elemento	Responsabilidad
Presentación	Navegador, Postman, frontend web	Consumir endpoints, enviar solicitudes y visualizar respuestas JSON.
Servicios	ORDS / Database Actions REST	Publicar módulos, plantillas y manejadores REST.
Lógica	PL/SQL	Registrar pagos, egresos, generar alícuotas, calcular deudas y procesar IA.
Datos	Oracle Database	Almacenar información relacional, JSON, auditoría y consultas IA.
Evidencia	GitHub y capturas	Versionar scripts, documentación y pruebas del proyecto.

7. Modelo de datos

El modelo de datos representa la realidad administrativa de una urbanización. La urbanización es la entidad general; dentro de ella existen viviendas; las viviendas están asociadas a propietarios; cada vivienda genera alícuotas por período; las alícuotas pueden pagarse; los pagos aumentan caja; los egresos disminuyen caja; y la auditoría registra operaciones importantes.

URBANIZACION

└─ VIVIENDA

└─ PROPIETARIO

└─ ALICUOTA

└─ PAGO

PERIODO

└─ ALICUOTA

EGRESO

PAGO



OPERACIONES IMPORTANTES — AUDITORIA_SISTEMA

PREGUNTAS NATURALES — CONSULTA_IA

7.1 Diccionario de tablas principales

Tabla	Función dentro del sistema	Uso operativo
URBANIZACION	Representa el conjunto residencial completo.	Base general para asociar viviendas.
PROPIETARIO	Registra datos personales y de contacto.	Responsable de una o más viviendas.
VIVIENDA	Representa casas, departamentos o locales.	Genera alícuotas mensuales.
PERIODO	Define mes y año de cobro.	Permite organizar alícuotas por tiempo.
ALICUOTA	Obligación mensual de una vivienda.	Puede estar pendiente, pagada, vencida o anulada.

Tabla	Función dentro del sistema	Uso operativo
PAGO	Pago realizado sobre una alícuota.	Genera ingreso y puede marcar alícuota como pagada.
EGRESO	Gasto administrativo de la urbanización.	Reduce el saldo de caja.
CAJA_MOVIMIENTO	Movimiento financiero de ingreso o egreso.	Permite calcular el saldo disponible.
AUDITORIA_SISTEMA	Bitácora de eventos relevantes.	Controla trazabilidad.
CONSULTA_IA	Historial de preguntas en lenguaje natural.	Guarda pregunta, interpretación y SQL generado.

7.2 Reglas de integridad aplicadas

- No se debe crear una vivienda si no existe el propietario asociado.
- No se debe crear una alícuota si no existe la vivienda ni el período correspondiente.
- No se debe registrar un pago si no existe la alícuota asociada.
- Los pagos y egresos no se eliminan físicamente para evitar descuadres financieros.
- La caja no se registra manualmente; se alimenta a partir de pagos y egresos.
- La auditoría se genera mediante triggers y no por inserción manual del usuario.

8. Implementación SQL y PL/SQL

8.1 Consultas avanzadas

Se implementaron consultas SQL para analizar deudores, recaudación mensual, saldo de caja, top de deudores, deuda por propietario, estado de alícuotas, ingresos y egresos. Estas consultas sirven como base para vistas, reportes REST y evidencias del proyecto.

Consulta	Finalidad	Resultado esperado
Deudores por filtros	Buscar propietarios que deben en meses específicos y por letra de apellido.	Listado filtrado por apellido y período.
Deuda por propietario	Calcular deuda total pendiente o vencida.	Ranking de deuda por persona.
Recaudación mensual	Agrupar pagos por mes y año.	Totales de ingresos por período.
Saldo de caja	Restar egresos a ingresos.	Saldo financiero actual.
Top deudores	Identificar propietarios con mayor deuda.	Priorización de cobro.
Reporte financiero JSON	Consolidar ingresos, egresos y saldo.	Salida estructurada para consumo REST.

8.2 Consulta principal solicitada

Una consulta central del proyecto permite responder a la necesidad: “Dame un listado de las personas que deben en septiembre y diciembre y que comiencen con el apellido A”. Esta consulta combina propietarios, viviendas, alícuotas, períodos y pagos para detectar deudas específicas.

```
SELECT
  p.id_propietario,
  p.nombres,
  p.apellidos,
  v.numero_vivienda,
  LISTAGG(pe.nombre_mes, ', ') WITHIN GROUP (ORDER BY pe.mes) AS meses_adeudados,
  SUM(a.valor - NVL(pg.total_pagado, 0)) AS deuda_total
```

```

FROM propietario p
INNER JOIN vivienda v ON v.id_propietario = p.id_propietario
INNER JOIN alicuota a ON a.id_vivienda = v.id_vivienda
INNER JOIN periodo pe ON pe.id_periodo = a.id_periodo
LEFT JOIN (
    SELECT id_alicuota, SUM(valor_pagado) AS total_pagado
    FROM pago
    GROUP BY id_alicuota
) pg ON pg.id_alicuota = a.id_alicuota
WHERE UPPER(p.apellidos) LIKE 'A%'
AND pe.mes IN (9, 12)
AND a.estado IN ('PENDIENTE', 'VENCIDA')
GROUP BY p.id_propietario, p.nombres, p.apellidos, v.numero_vivienda
HAVING COUNT(DISTINCT pe.mes) = 2
ORDER BY p.apellidos, p.nombres;

```

8.3 Procedimientos y funciones PL/SQL

Objeto PL/SQL	Tipo	Descripción
fn_deuda_vivienda	Función	Calcula la deuda pendiente de una vivienda.
fn_deuda_propietario	Función	Calcula la deuda total asociada a un propietario.
pr_registrar_pago	Procedimiento	Registra un pago, actualiza alícuota y genera movimiento de caja.
pr_generar_alicutas_mensuales	Procedimiento	Genera alícuotas para todas las viviendas en un mes determinado.
pr_actualizar_vencidas	Procedimiento	Marca como vencidas las alícuotas pendientes fuera de fecha.
pr_registrar_egreso	Procedimiento	Registra un gasto de la urbanización y su salida en caja.
pr_consulta_lenguaje_natural	Procedimiento	Procesa preguntas en lenguaje natural y guarda SQL generado.

8.4 Triggers y auditoría

Los triggers se utilizaron para controlar eventos importantes y mantener evidencia. Oracle permite crear triggers mediante la sentencia CREATE TRIGGER o desde herramientas gráficas como SQL Developer [3].

Trigger	Evento auditado	Resultado
trg_aud_estado_alicuota	Cambio de estado de alícuota.	Registra estado anterior y nuevo en auditoría.
trg_aud_pago_insert	Inserción de pago.	Registra evento de pago realizado.
trg_aud_egreso_insert	Inserción de egreso.	Registra salida de dinero.
trg_validar_pago	Validación previa de pago.	Normaliza método de pago y valida valores.

9. API REST con ORDS y Database Actions

La API se publicó con el módulo ORDS mod_alicutas_api y el prefijo /api/. En Oracle Database Actions, el módulo se visualiza dentro de REST > Módulos, mostrando las plantillas URI y los manejadores asociados. La documentación de Oracle indica que Database Actions permite crear servicios REST mediante páginas de módulos, plantillas y manejadores [2].

Aunque en clase se puede crear el CRUD desde la interfaz web, en este proyecto los servicios fueron generados principalmente por scripts ORDS. Esta decisión permite versionar el trabajo en GitHub y repetir la instalación del proyecto en otro entorno.

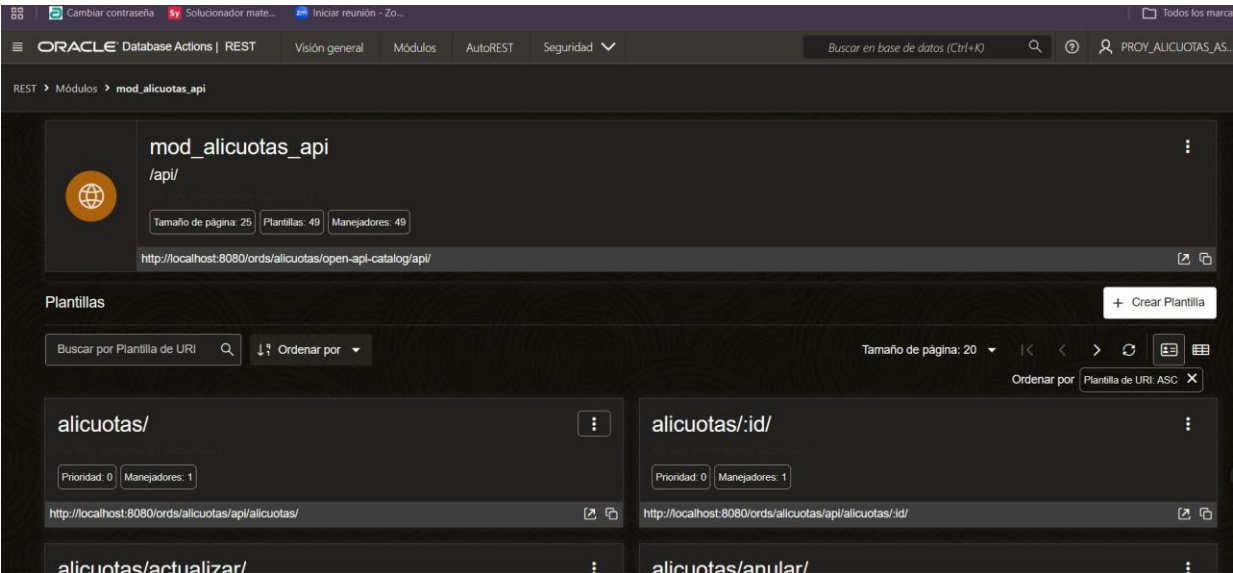


Figura 1. Módulo mod_alicutas_api visualizado desde Oracle Database Actions REST.

9.1 Diferencia entre interfaz REST y endpoint API

Ruta	Función
http://localhost:8080/ords/	Entrada a ORDS / Database Actions / inicio de sesión.
http://localhost:8080/ords/alicutas/api/info/	Endpoint informativo de la API REST.
http://localhost:8080/ords/alicutas/api/propietarios/	Endpoint para listar propietarios.
http://localhost:8080/ords/alicutas/api/deudores/filtro A/9/12/	Endpoint para consulta principal de deudores.
http://localhost:8080/ords/alicutas/api/frontend/	Frontend básico servido desde ORDS, si se habilita.

9.2 Endpoints principales

Módulo	Método	Endpoint	Descripción
Info	GET	/info/	Comprueba que la API se encuentra activa.
Propietarios	GET	/propietarios/	Lista propietarios.
Propietarios	POST	/propietarios/crear/	Crea propietario mediante JSON.
Propietarios	PUT	/propietarios/actualizar/	Actualiza propietario.
Propietarios	DELETE	/propietarios/eliminar/	Inactiva propietario.
Viviendas	GET	/viviendas/	Lista viviendas.
Viviendas	POST	/viviendas/crear/	Crea vivienda.
Períodos	POST	/periodos/crear/	Crea período mensual.
Alícuotas	POST	/alicutas/generar-mensual/	Genera alícuotas para todas las viviendas.
Pagos	POST	/pagos/registrar/	Registra pago de alícuota.
Egresos	POST	/egresos/registrar/	Registra gasto de la urbanización.
IA	POST	/ia/preguntar/	Procesa pregunta de lenguaje natural.
Reportes	GET	/deudores/filtroA/9/12/	Consulta principal de deudores.

10. CRUD y consumo de servicios

El CRUD fue implementado con operaciones GET, POST, PUT y DELETE. Para operaciones financieras sensibles, DELETE no representa una eliminación física, sino una acción lógica como inactivar, cerrar o anular. Esta decisión preserva la integridad histórica del sistema.

Entidad	Ver	Crear	Actualizar	Eliminar lógico
Propietario	GET /propietarios/	POST /propietarios/crear/	PUT /propietarios/actualizar/	DELETE /propietarios/eliminar/
Vivienda	GET /viviendas/	POST /viviendas/crear/	PUT /viviendas/actualizar/	DELETE /viviendas/eliminar/
Período	GET /periodos/	POST /periodos/crear/	PUT /periodos/actualizar/	DELETE /periodos/eliminar/
Alícuota	GET /alicuotas/	POST /alicuotas/crear/	PUT /alicuotas/actualizar/	DELETE /alicuotas/anular/
Pago	GET /pagos/	POST /pagos/registrar/	PUT /pagos/actualizar/	No se elimina físicamente
Egreso	GET /egresos/	POST /egresos/registrar/	PUT /egresos/actualizar/	No se elimina físicamente

10.1 Ejemplo de creación de propietario

POST http://localhost:8080/ords/alicuotas/api/propietarios/crear/
Content-Type: application/json

```
{
  "cedula": "0109999999",
  "nombres": "Juan",
  "apellidos": "Arias",
  "telefono": "0999999999",
  "correo": "juan.arias@gmail.com"
}
```

10.2 Ejemplo de registro de pago

POST http://localhost:8080/ords/alicuotas/api/pagos/registrar/
Content-Type: application/json

```
{
  "id_alicuota": 10,
  "valor_pagado": 45,
  "metodo_pago": "EFECTIVO",
  "referencia": "REC-001"
}
```

Al registrar el pago, el procedimiento PL/SQL inserta en PAGO, registra el ingreso en CAJA_MOVIMIENTO, actualiza la alícuota si el valor cubre la deuda y deja trazabilidad mediante triggers.

11. Módulo IA/NLQ

El módulo IA/NLQ se implementó para demostrar el flujo de consultas en lenguaje natural sin depender de Oracle Cloud. En lugar de usar Select AI real, el sistema usa funciones y procedimientos PL/SQL para interpretar palabras clave y generar una consulta SQL asociada. Oracle Select AI sirve

como referencia conceptual porque permite interactuar con la base mediante prompts de lenguaje natural para generar, ejecutar o explicar SQL [9].

Elemento	Implementación
Entrada	Pregunta escrita por el usuario en lenguaje natural.
Interpretación	Búsqueda de palabras clave como deudores, saldo, recaudación, pagos, egresos o apellido A.
Salida	SQL generado automáticamente según la intención detectada.
Evidencia	Registro en la tabla CONSULTA_IA con pregunta, interpretación, SQL, estado y fecha.
Consumo	Endpoint POST /ia/preguntar/ y GET /ia/consultas/.

11.1 Ejemplo de pregunta IA

POST http://localhost:8080/ords/alicuotas/api/ia/preguntar/

Content-Type: application/json

```
{
  "pregunta": "Dame un listado de las personas que deben en septiembre y diciembre y que comiencen con el apellido A"
}
```

11.2 Flujo del módulo IA/NLQ

Pregunta del usuario

↓

FN_INTERPRETACION_IA detecta la intención

↓

FN_GENERAR_SQL_IA arma el SQL relacionado

↓

PR_CONSULTA LENGUAJE NATURAL guarda la evidencia

↓

CONSULTA_IA conserva pregunta, SQL, estado y fecha

↓

ORDS devuelve respuesta JSON al cliente

11.3 Justificación de la IA simulada

El entorno local no dispone de perfil activo de Select AI ni cuenta Oracle Cloud configurada con proveedor de IA. Por ello, se implementó una versión académica y funcional que conserva el comportamiento principal esperado: transformar una intención escrita en lenguaje natural en una consulta SQL relacionada y dejar evidencia del proceso. Esto permite defender la integración de IA sin depender de servicios externos.

12. Frontend web y Postman

El consumo principal de la API fue validado con Postman, herramienta que permite crear y enviar solicitudes API e inspeccionar la respuesta devuelta por el servidor [7]. Adicionalmente, se preparó un frontend básico en HTML, CSS y JavaScript para demostrar una interfaz de usuario simple.

Cuando el frontend se abre como archivo local, el navegador puede bloquear peticiones por políticas CORS. Por ello, se propuso servir el frontend desde ORDS en la ruta /frontend/, de manera que la página y los endpoints pertenezcan al mismo origen.

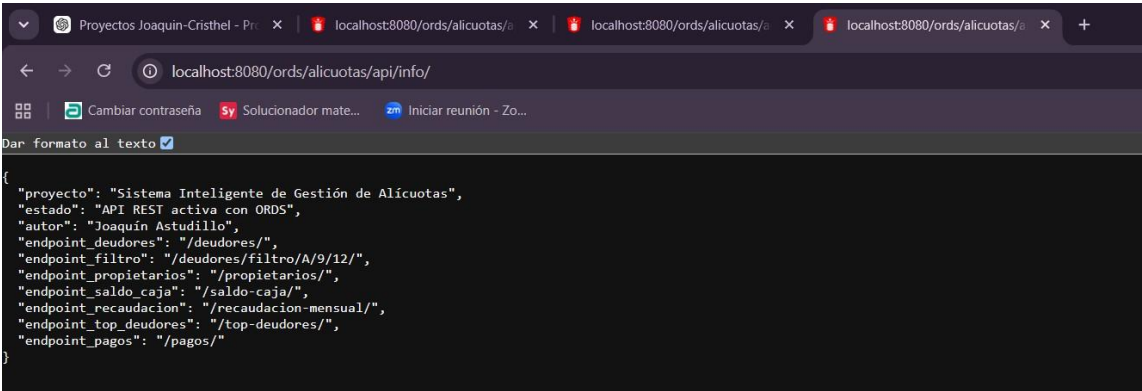


Figura 2. Endpoint /api/info/ devolviendo JSON de estado de la API.

Herramienta	Uso en el proyecto	Evidencia
Postman	Prueba de GET, POST, PUT y DELETE.	Capturas de respuesta HTTP 200 y JSON.
Navegador	Validación de endpoints GET.	Captura de /info/ y reportes.
Frontend básico	Interfaz para consumir operaciones principales.	Captura de formularios y respuestas.
Database Actions	Visualización de módulos, plantillas y manejadores REST.	Captura del módulo mod_alicuotas_api.

13. Gestión del repositorio GitHub

El proyecto se subió a GitHub para organizar scripts, evidencias, plantillas Postman, documentación y README. GitHub permite agregar archivos desde el navegador usando la opción Add file > Upload files y confirmar los cambios con un commit [8].

Carpeta	Contenido esperado
Base de Datos SQL	Scripts de creación, inserts, consultas, PL/SQL, triggers, JSON, ORDS e IA.
Postman	Plantillas de endpoints y cuerpos JSON de prueba.
Evidencias	Capturas de Postman, navegador, ORDS, Database Actions, GitHub e IA.
Documentación	Informe final, manual técnico y anexos.
README.md	Descripción del proyecto, tecnologías, endpoints, flujo y autor.

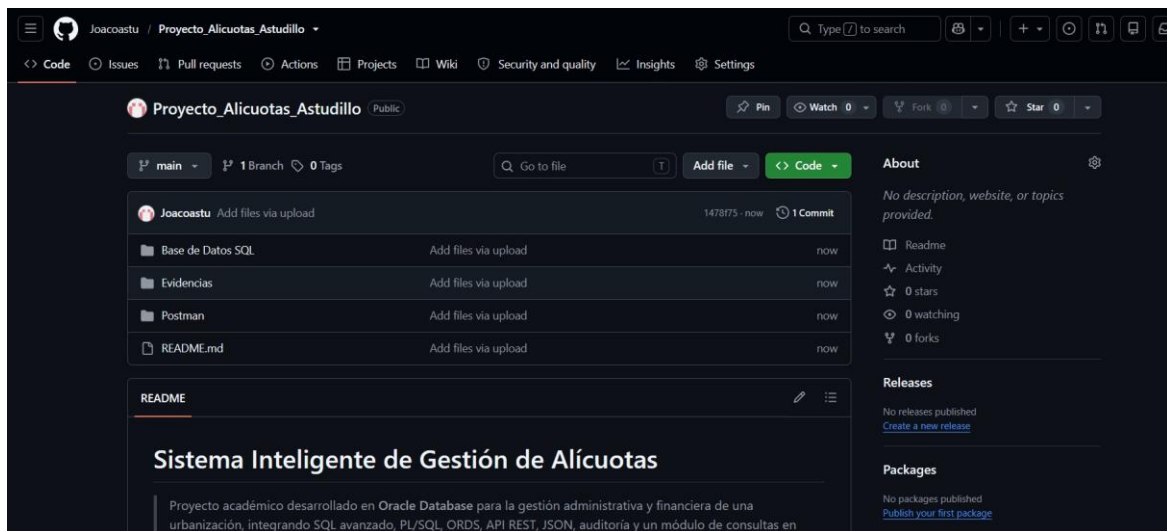


Figura 3. Repositorio GitHub con carpetas, README y evidencias iniciales.

14. Evidencias y capturas

En esta sección se deben insertar las capturas finales del proyecto. Se recomienda mantener una evidencia por funcionalidad clave, con título, descripción y resultado esperado.

N.º	Evidencia sugerida	Descripción	Estado
1	API /info/ funcionando	Muestra proyecto, estado, autor y endpoints principales.	Insertar captura
2	Database Actions REST	Módulo mod_alicuotas_api con plantillas y manejadores.	Insertada / actualizar si aplica
3	GET propietarios	Lista propietarios registrados.	Insertar captura
4	POST crear propietario	Crea propietario desde JSON.	Insertar captura
5	PUT actualizar propietario	Actualiza datos del propietario.	Insertar captura
6	DELETE inactivar propietario	Cambia estado a INACTIVO.	Insertar captura
7	POST registrar pago	Registra pago y actualiza caja.	Insertar captura
8	POST registrar egreso	Registra gasto y reduce caja.	Insertar captura
9	GET saldo caja	Muestra ingresos, egresos y saldo.	Insertar captura
10	POST IA preguntar	Pregunta natural y SQL generado.	Insertar captura

```

id_propietario : 4,
"apellidos": "Aguilar",
"nombres": "Diana",
"telefono": "0990000004",
"correo": "diana.aguilar4@correo.com",
"id_vivienda": 4,
"numero_vivienda": "A-04",
"manzana": "A",
"mes": 3,
"anio": 2026,
"id_alicuota": 39,
"valor": 45,
"estado": "VENCIDA",
"fecha_limite": "2026-03-10T00:00:00Z"

```

GET Send

Docs Params Authorization Headers 7 Body **Scripts** Settings Cookies

Pre-request

Post-response

1 Use JavaScript to write tests, visualize response, and more.

Packages </> Snippets

Body Cookies Headers 3 Test Results 200 OK • 17.26 s • 7.26 KB • Save Response

{ } JSON Preview Visualize

```
65     "fecha_limite": "2026-09-10T00:00:00Z"
66   },
67   {
68     "id_propietario": 4,
69     "apellidos": "Aguilar",
70     "nombres": "Diana",
71     "telefono": "0990000004",
72     "correo": "diana.aguilar4@correo.com",
73     "id_vivienda": 4,
74     "numero_vivienda": "A-04",
75     "manzana": "A",
76     "mes": 12,
77     "anio": 2026,
```

15. Pruebas realizadas

Las pruebas se realizaron combinando SQL Developer, Postman, navegador, Database Actions y GitHub. Cada prueba verificó que el endpoint existiera, aceptara el método HTTP correcto, procesara parámetros o JSON y devolviera una respuesta válida.

Prueba	Entrada	Resultado esperado	Validación
Conexión ORDS	Abrir /ords/	ORDS activo.	Correcto si carga Database Actions.
Info API	GET /info/	JSON de presentación.	Código 200 OK.
Listar propietarios	GET /propietarios/	Arreglo de propietarios.	JSON con items.
Crear propietario	POST JSON	Nuevo registro en PROPIETARIO.	Verificar con GET.
Actualizar propietario	PUT JSON	Datos modificados.	Verificar por ID.
Inactivar propietario	DELETE JSON	Estado INACTIVO.	Verificar por ID.
Generar alícuotas	POST mes, año, valor	Alícuotas para viviendas.	Verificar en ALICUOTA.
Registrar pago	POST id_alicuota, valor	Pago, caja y estado actualizados.	Verificar PAGO y CAJA.
Registrar egreso	POST categoría, valor	Egreso y salida de caja.	Verificar EGRESO y CAJA.
Consulta IA	POST pregunta	SQL generado guardado.	Verificar CONSULTA_IA.

16. Riesgos y limitaciones

Riesgo o limitación	Impacto	Tratamiento aplicado
Select AI real no configurado	No se usa IA nativa de Oracle Cloud.	Se implementó módulo NLQ local en PL/SQL.
CORS en navegador	Frontend local puede fallar con Failed to fetch.	Se recomienda Postman o frontend servido desde ORDS.
Endpoint base /api sin ruta	Puede mostrar 404 al entrar	Se usa /api/info/ como portada de la

Riesgo o limitación	Impacto	Tratamiento aplicado
	directamente.	API.
Eliminación de pagos/egresos	Podría descuadrar caja.	Se evita eliminación física y se conserva trazabilidad.
Datos de prueba	No representan una urbanización real.	Se usan como dataset académico controlado.
Credenciales	Riesgo de exposición en capturas.	Se recomienda ocultar contraseñas y no subir wallets o claves.

17. Conclusiones

Se implementó un sistema completo de gestión de alícuotas que cubre propietarios, viviendas, períodos, alícuotas, pagos, egresos, caja, auditoría, reportes, API REST y consultas en lenguaje natural.

El uso de ORDS permitió exponer la lógica de Oracle Database como servicios REST consumibles desde Postman, navegador y un frontend básico. Esto demuestra una integración real entre base de datos y aplicaciones externas.

La creación de endpoints mediante scripts ORDS resultó más controlada y reproducible que hacerlo únicamente desde la interfaz gráfica, ya que los scripts pueden versionarse y reutilizarse en GitHub.

El módulo IA/NLQ implementado en PL/SQL permitió simular el flujo de lenguaje natural a SQL sin depender de Oracle Cloud. Aunque no reemplaza a Select AI real, sí cumple la finalidad académica de demostrar interpretación, generación de SQL y almacenamiento de evidencia.

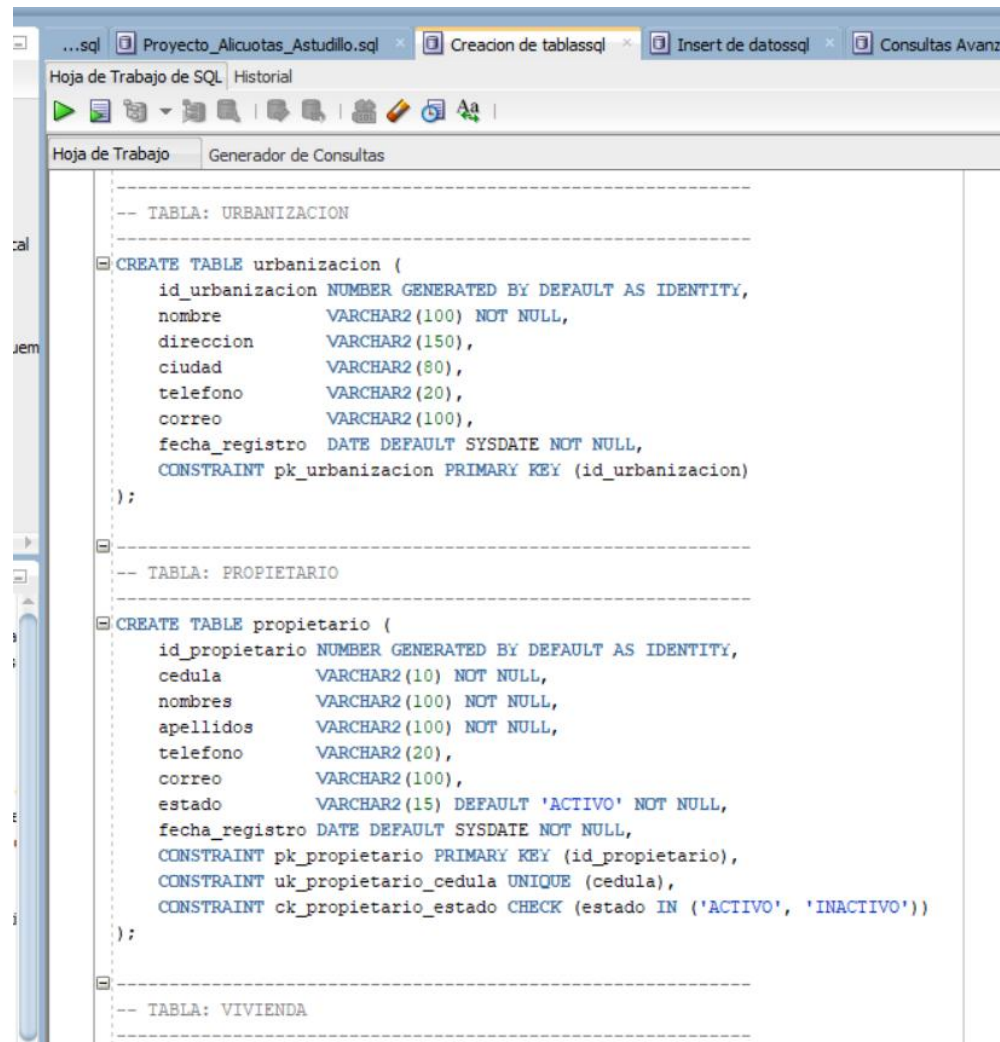
La auditoría mediante triggers y la eliminación lógica de datos financieros fortalecen la trazabilidad del sistema, evitando pérdida de historial o descuadres en caja.

El repositorio GitHub mejora la presentación del proyecto, ya que centraliza scripts, documentación, README, evidencias y plantillas de prueba.

18. Bibliografía

- [1] Oracle. (2026). Introduction to Oracle REST Data Services. Oracle Documentation.
<https://docs.oracle.com/en/database/oracle/oracle-rest-data-services/25.3/orddg/introduction-to-Oracle-REST-Data-Services.html>
- [2] Oracle. (2026). Creating RESTful Web Services using Database Actions. Oracle SQL Developer Web Documentation. <https://docs.oracle.com/en/database/oracle/sql-developer-web/20.3/sdweb/creating-restful-services.html>
- [3] Oracle. (2026). Creating Triggers. Oracle Database Documentation.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/tdddg/creating-triggers.html>
- [4] Oracle. (2026). Overview of PL/SQL. Oracle Database PL/SQL Language Reference.
<https://docs.oracle.com/en/database/oracle/oracle-database/21/lpls/overview.html>
- [5] Oracle. (2026). SQL/JSON Function JSON_OBJECT. Oracle Database JSON Developer's Guide.
https://docs.oracle.com/en/database/oracle/oracle-database/26/adjsn/sql-json-function-json_object.html
- [6] Oracle. (2026). JSON_ARRAYAGG. Oracle Database SQL Language Reference.
https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/JSON_ARRAYAGG.html
- [7] Postman. (2026). Send API requests and get response data in Postman. Postman Learning Center.
<https://learning.postman.com/docs/use/send-requests/requests/>
- [8] GitHub. (2026). Adding a file to a repository. GitHub Docs. <https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository>
- [9] Oracle. (2026). About Select AI. Oracle Database Select AI Documentation.
<https://docs.oracle.com/en/database/oracle/oracle-database/26/selai/select-ai-about.html>

Anexo A. Plantilla para insertar capturas adicionales



The screenshot shows a SQL IDE window with the title bar containing several tabs: "...sql", "Proyecto_Alcuotas_Astudillo.sql", "Creacion de tablassql", "Insert de datossql", and "Consultas Avanz". The main window has a menu bar with "Hoja de Trabajo de SQL" and "Historial". Below the menu bar is a toolbar with various icons. The main area is divided into two tabs: "Hoja de Trabajo" and "Generador de Consultas". The "Hoja de Trabajo" tab is active and displays a SQL script. The script is organized into sections separated by dashed lines, each starting with a comment indicating the table name. The first section is for the "URBANIZACION" table, the second for the "PROPIETARIO" table, and the third for the "VIVIENDA" table. Each section contains a "CREATE TABLE" statement with column definitions and constraints.

```
-- TABLA: URBANIZACION
-----
CREATE TABLE urbanizacion (
    id_urbanizacion NUMBER GENERATED BY DEFAULT AS IDENTITY,
    nombre          VARCHAR2(100) NOT NULL,
    direccion       VARCHAR2(150),
    ciudad          VARCHAR2(80),
    telefono        VARCHAR2(20),
    correo          VARCHAR2(100),
    fecha_registro  DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_urbanizacion PRIMARY KEY (id_urbanizacion)
);

-- TABLA: PROPIETARIO
-----
CREATE TABLE propietario (
    id_propietario  NUMBER GENERATED BY DEFAULT AS IDENTITY,
    cedula          VARCHAR2(10) NOT NULL,
    nombres         VARCHAR2(100) NOT NULL,
    apellidos       VARCHAR2(100) NOT NULL,
    telefono        VARCHAR2(20),
    correo          VARCHAR2(100),
    estado          VARCHAR2(15) DEFAULT 'ACTIVO' NOT NULL,
    fecha_registro  DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_propietario PRIMARY KEY (id_propietario),
    CONSTRAINT uk_propietario_cedula UNIQUE (cedula),
    CONSTRAINT ck_propietario_estado CHECK (estado IN ('ACTIVO', 'INACTIVO'))
);

-- TABLA: VIVIENDA
-----
```

```
-- 03_CONSULTAS_AVANZADAS.sql
-- Proyecto: Sistema Inteligente de Gestión de Aícuotas
```

```
-----
-- CONSULTA 1
-- Personas que deben en septiembre y diciembre
-- y cuyo apellido inicia con la letra A.
-----
```

```
SELECT
    p.apellidos,
    p.nombres,
    v.numero_vivienda,
    pe.mes,
    pe.ano,
    a.valor,
    a.estado
FROM propietario p
JOIN vivienda v ON p.id_propietario = v.id_propietario
JOIN alicuota a ON v.id_vivienda = a.id_vivienda
JOIN periodo pe ON a.id_periodo = pe.id_periodo
WHERE a.estado IN ('PENDIENTE', 'VENCIDA')
    AND pe.mes IN (9, 12)
    AND UPPER(p.apellidos) LIKE 'A%'
ORDER BY p.apellidos, pe.mes;
```

```
-----
-- CONSULTA 2
-- Total de deuda acumulada por propietario.
-----
```

```
SELECT
    p.id_propietario,
    p.apellidos || ' ' || p.nombres AS propietario,
    v.numero_vivienda,
```

```
-- 04_INDICES_OPTIMIZACION.sql
-- Índices para mejorar consultas frecuentes
```

```
CREATE INDEX idx_propietario_apellidos
ON propietario(apellidos);
```

```
CREATE INDEX idx_alicuenta_estado
ON alicuenta(estado);
```

```
CREATE INDEX idx_alicuenta_periodo
ON alicuenta(id_periodo);
```

```
CREATE INDEX idx_alicuenta_vivienda
ON alicuenta(id_vivienda);
```

```
CREATE INDEX idx_pago_alicuenta
ON pago(id_alicuenta);
```

```
CREATE INDEX idx_pago_fecha
ON pago(fecha_pago);
```

```
CREATE INDEX idx_caja_tipo_fecha
ON caja_movimiento(tipo_movimiento, fecha_movimiento);
```

```
CREATE INDEX idx_vivienda_manzana
ON vivienda(manzana);
```

```
EXPLAIN PLAN FOR
```

```
SELECT
```

```
    p.apellidos,
```

```
-- 06_TRIGGERS_AUDITORIA.sql
-- Proyecto: Sistema Inteligente de Gestión de Alicuotas
-- Estudiante: Joaquín Astudillo

-----

-- TABLA DE AUDITORÍA

-----

CREATE TABLE auditoria_sistema (
    id_auditoria    NUMBER GENERATED BY DEFAULT AS IDENTITY,
    tabla_afectada  VARCHAR2(50) NOT NULL,
    operacion       VARCHAR2(20) NOT NULL,
    id_registro     NUMBER,
    detalle         VARCHAR2(500),
    usuario_bd      VARCHAR2(100) DEFAULT USER,
    fecha_auditoria DATE DEFAULT SYSDATE,
    CONSTRAINT pk_auditoria_sistema PRIMARY KEY (id_auditoria)
);

-----

-- TRIGGER 1: AUDITAR CAMBIO DE ESTADO DE ALÍCUOTAS

-----

CREATE OR REPLACE TRIGGER trg_aud_estado_alicuota
AFTER UPDATE OF estado ON alicuota
FOR EACH ROW
WHEN (OLD.estado <> NEW.estado)
BEGIN
    INSERT INTO auditoria_sistema (
        tabla_afectada,
        operacion,
        id_registro,
        detalle
    ) VALUES (
```